

Introduction to High Performance Computing with Julia

Adeleke O. Bankole



Data Science Africa Summer School
Makerere University, Kampala, Uganda
Tuesday, 21 July 2026

What is High Performance Computing (HPC)?

- A computing resource $\gg$ one provided by one laptop or server
- **Clusters**: groups of interconnected computers, servers, storage, etc
- **Supercomputers**: most powerful/performant computers in the world
- **Cloud computing**: on-demand access to computing resources over the internet

Applications of HPC include:

- Weather forecasting, climate modelling
- ML/AI
- Big data analytics
- CFD and MD simulations

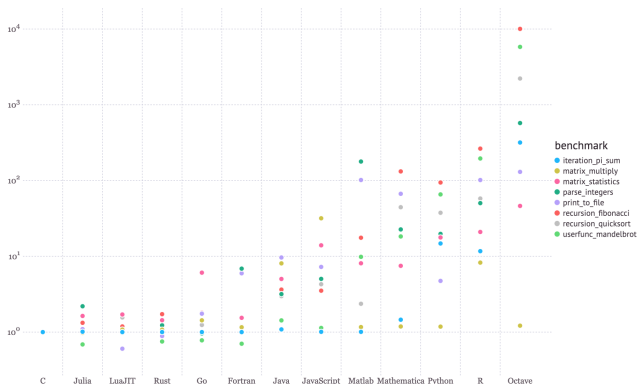
Our aim: give you a concise introduction to HPC and Julia

Entrance via Nodes

- A **node** is a single computer in a cluster or supercomputer
- Each node has its own CPU/GPU, memory, and storage
- Nodes are connected via a high-speed network (Infiniband, Slingshot)
- Nodes can be used to run parallel computations
- **Login nodes**: accessing the system and submitting jobs
- **Compute nodes**: running actual computations

Do not run compute jobs/heavy computations on login nodes!

Why Julia for HPC?



- Julia can be **fast** as C/C++ and Fortran, while being as easy to write as Python or R
- Julia can be **parallel**, serial and parallel code performance on par with C/C++ and Fortran

Why Julia for HPC?

- Solves the **two-language problem**: write high-level code and get low-level performance



- Julia can be **inclusive**: has simple syntax that is easy to learn and use
- Julia can be **portable** across different architectures and platforms

Shared memory computing: Multi-threading

- Idea from **openMP** - a **fork-join** model
- **Threads** are a way to achieve parallelism within a single process
- **OhMyThreads.jl** is one of the Julia package for working with threads
- Threads allow for communication between threads running on the same node

Distributed Computing: MPI and MPI.jl

- **MPI** (Message Passing Interface) is a standard for distributed computing, i.e, OpenMPI, MPICH
- **MPI.jl** is the Julia package for working with MPI
- MPI allows communication between processes running on diff nodes using MPI

```
MPI.Init()
```

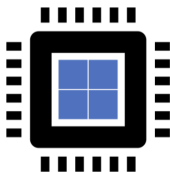
```
comm    = MPI.COMM_WORLD
rank    = MPI.Comm_rank(comm)
nranks  = MPI.Comm_size(comm)
```

```
println("Hello world, I am rank $rank of $nranks")
```

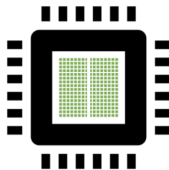
```
MPI.Finalize()
```

Run with: `mpiexecjl --project -np 4 julia mpi_hello.jl`

CPU vs GPU



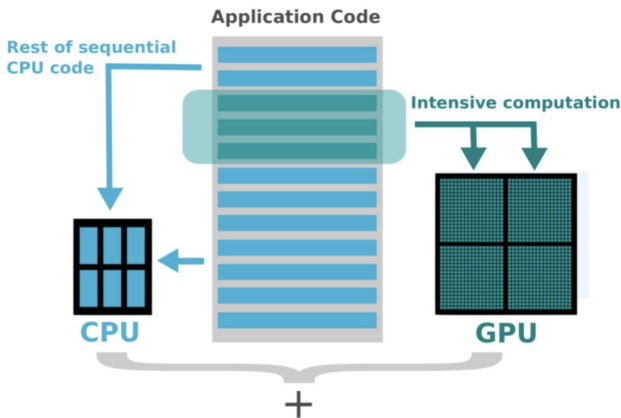
- General workhorse
- Contains dozens of cores
- Good for serial processing and tasks that require low latency



- Specialized
- Contains thousands of cores
- Good for parallel processing and tasks that require high throughput

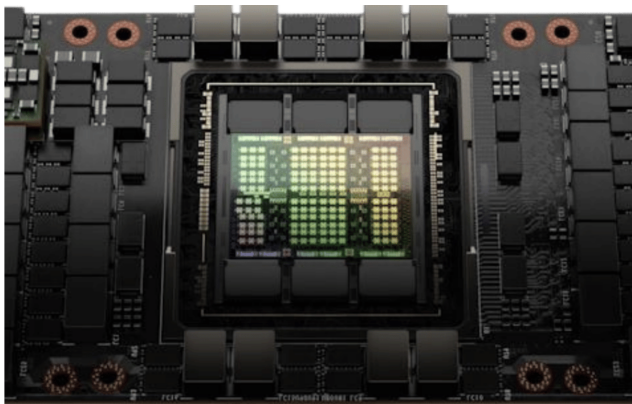
Heterogeneous Systems

GPU acceleration follows an **offloading model**



- Achieve speed by **porting** computationally intensive work to GPUs
- Data needs to be copied from CPU to GPU, perform computations on the GPU, and output is sent back to the CPU

NVIDIA HGX H200 GPU



- Optimized for AI and HPC
- Scalability
- Enhanced thermal efficiency

GPU computing in Julia

Vendor specific packages

- **CUDA.jl** for NVIDIA

```
using CUDA

function saxpy!(a,X,Y)
    i = (blockIdx().x - 1) * blockDim().x + threadIdx().x
    if i <= length(Y)
        @inbounds Y[i] = a * X[i] + Y[i]
    end
    return nothing
end

@cuda threads=32 blocks=cld(length(Y), 32) saxpy!(a, X, Y)
```

- **AMDGPU.jl** for AMD
- **oneAPI.jl** for Intel
- **Metal.jl** for macOS

GPU computing in Julia

Some other core packages

- **KernelAbstractions.jl** for vendor agnostic kernel programming

```
using KernelAbstractions
using CUDA

@kernel function saxpy!(a, @Const(X), Y)
    I = @index(Global)
    @inbounds Y[I] = a * X[I] + Y[I]
end

saxpy!(CUDABackend())(a, X, Y, ndrange=length(Y))
```

- **GPUArrays.jl** for GPU array programming
- **GPUCompiler.jl** for compiling native Julia code to GPU kernels
- **Adapt.jl** for converting complex structures from CPU to GPU.

Stay in touch:

 **Email:** adeleke.bankole@gmail.com, ab3191@cam.ac.uk

 **Website:** adelekebankole.github.io

 **GitHub:** [AdelekeBankole](https://github.com/AdelekeBankole)